PDF → Multimodal Retrieval Index

Technical report Author: Navin Kumar

Live app: https://relevant-section-identification-qqgcuwkcmq-uc.a.run.app **Source:** https://github.com/ns-0437/relevant-section-identification

---

1. Objective

Task: given a PDF and a natural-language query, identify the pages and section headings containing information relevant to answering the query. Return the page number(s) and, where possible, the corresponding section heading(s).

The approach taken is to turn the PDF into retrieval-ready chunks that preserve three different kinds of content — prose, tables, and figures — embed them, and aggregate chunk hits back up to a ranked list of pages.

Pipeline requirements as specified:

#	Requirement
R1	Partition with `unstructured`, `hi_res` strategy, extracting tables and images
R2	Chunk text by title: min 1000, soft 3000, hard 5000 characters
R3	~500 characters of overlap between chunks
R4	Page number and section title in every chunk's metadata
R5	Images: base64 in metadata, captioned by a local model, OCR keywords (stopwords removed) appended
R6	Tables: converted to markdown; caption first; split with headers repeated if over 5000 chars
R7	Embed with a small (<400M) current model, store in ChromaDB

All seven are implemented. Section 6 reports which are verified against real data and which are only covered by synthetic tests.

---

2. Test document

File	`relevant_section_identification-sample.pdf`
Content	Analog Devices / Maxim **MAX77751** 3.15A USB-C autonomous charger datasheet
Pages	40
Size	1,819,441 bytes
SHA-256	`0C7121C956C97EC5CFB809D388631D1473B8208A25DD4F84FE3E790AB57471B1`

Verified byte-identical to `AI Projects/relevant-section-identification/data/sample.pdf`.

A datasheet is a deliberately hard case: dense multi-column specification tables, dot-leader rows, oscilloscope traces, mechanical drawings, and rotated axis labels.

---

3. Environment

Component	Version
Python	3.11.9 (venv at `C:\Documents2\resume-optimizer\.venv`)
unstructured / unstructured-inference	0.25.2 / 1.6.13
transformers / sentence-transformers	5.9.0 / 5.5.1
torch	2.13.0+**cu126**
chromadb	1.5.9
Tesseract OCR	5.4.0
Poppler	25.07
GPU	NVIDIA RTX 3050 Laptop, **4 GB VRAM**, driver 555.97

Two environment issues worth recording:

- **CUDA wheel selection.** `pip install --upgrade torch --index-url .../cu124` silently no-ops when the index's newest torch is older than the installed CPU build. The exact variant must be pinned: `torch==2.13.0+cu126`. CUDA 13 wheels need a 580+ driver and are unusable here.
- **Binary discovery.** Tesseract was installed but absent from `PATH`. Since both `pytesseract` and `unstructured`'s OCR shell out to the binary, the pipeline patches `PATH` in-process at import (`ensure_binaries_on_path()`) rather than requiring a system change.

---

4. Architecture

```
PDF
 └─ partition_pdf(strategy="hi_res", infer_table_structure=True,
                  extract_image_block_to_payload=True)          → 852 elements
    ├─ text elements  (787) ─→ chunk_by_title(1000/3000/5000, overlap=500)  → 53 text chunks
    ├─ Table elements  (19) ─→ HTML → markdown, caption first, split+repeat  → 19 table chunks
    └─ Image elements  (46) ─→ LLaVA caption + Tesseract OCR → keywords       → 46 image chunks
                                                                  │
                          sort by page, apply cross-chunk overlap ▼
                                                          118 chunks (JSON)
                                                                  │
                             EmbeddingGemma-300m, 768-dim, cosine ▼
                                                          ChromaDB collection
```

4.1 Text

`chunk_by_title` with `combine_text_under_n_chars=1000`, `new_after_n_chars=3000`, `max_characters=5000`, `overlap=500`, `overlap_all=True`.

`overlap_all=True` is the load-bearing flag: without it `unstructured` applies overlap only when splitting an oversized element, not between adjacent chunks.

The section title is recovered from each chunk's `orig_elements`, falling back to the last-seen `Title` so chunks that continue a section inherit its heading.

4.2 Tables — read from the text layer, not OCR

`unstructured`'s `infer_table_structure=True` populates `metadata.text_as_html`, but it produces that HTML by **OCRing a rendered image of the table region**. On a digital PDF this discards a perfectly good text layer. The measured cost on this document was severe:

	OCR'd HTML	PDF text layer
Page 5, row 3	`Battery Only Quiescent | | USB Type-C as UFP and BATT = SYS = | | |`	`Battery Only Quiescent Current | IBATT_Q | USB Type-C as UFP and BATT = SYS = 3.6V | 30 50 | µA`

The entire SYMBOL column and every numeric limit were lost. `IBATT_Q` did not appear **anywhere** in the 118 chunks of the first build, so no amount of retrieval tuning could ever surface it.

Tables are therefore extracted with **pdfplumber from the embedded text layer**, matched positionally to `unstructured`'s Table elements within each page, with the OCR'd HTML kept as a fallback. The source used is recorded per chunk in `metadata.table_source`. On this document: **17 of 19 tables from the text layer**, 2 falling back to OCR.

Markdown rendering is unchanged: row 0 becomes the header, rows are padded to the widest row, literal pipes escaped as `\|`.

Captions are found by scanning up to two elements either side of the table on the same page for a `FigureCaption` or text matching `^(table|tbl|exhibit)`.

Splitting: when a table exceeds 5000 characters it is divided on row boundaries, each part re-emitting the caption and the header + separator rows, and carrying the trailing rows of the previous part (up to ~500 chars) as overlap. Splits land on row boundaries, so overlap rounds *down* to whole rows — never mid-row, which would corrupt the markdown.

4.3 Images

Base64 comes straight from `unstructured`'s payload. Each image is captioned by a local LLaVA model and OCR'd with Tesseract; OCR text is tokenised, stopwords removed (NLTK, with a built-in fallback list), and the keywords appended to the caption. The chunk text is `caption + "\n\nKeywords: " + keywords`; the base64 lives in `metadata.image_base64`.

Model choice. The default is `llava-hf/llava-1.5-7b-hf`, but 4 GB of VRAM cannot hold it even in 4-bit once the vision tower and activations are resident. This run used **`llava-hf/llava-interleave-qwen-0.5b-hf`**, whose captions are unreliable on technical figures (§7.1).

Composition of an image chunk (v3). Because the generated caption is the least trustworthy element, it is ordered last and trimmed — and *proportion* matters more than order, since it was ~570 characters against ~44 of OCR keywords, i.e. ~90% of the embedded text:

1. the document's **own figure caption** (`Figure 12. System Configuration for Temperature Management`), located by scanning neighbouring elements for a `FigureCaption` or text matching `^Figure\s*\d` — real author-written text, recovered for **30 of 46** images;
2. **OCR keywords**, stopwords removed;
3. the **generated caption, first sentence only** — and dropped entirely for the 10 images with neither a figure caption nor OCR keywords, where it would be the chunk's only content with nothing to anchor it. Those fall back to `Figure in section: {title}`.

The full generated caption is always retained in `metadata.caption`. Median image-chunk length fell from 611 to 191 characters. Effect on retrieval: `multi` queries hit@1 25% → 75% and nDCG 0.314 → 0.629; `figure` R@5 83% → 100%.

4.4 Cross-chunk overlap

`chunk_by_title` handles text→text overlap. Table splits carry their own row overlap. A final pass gives the remaining chunks — every image chunk and the first part of every table — a 500-character prefix taken from the preceding chunk, preserving the un-prefixed version in `metadata.core_text`.

4.5 Boilerplate stripping

Every one of the 40 pages carries the running header `MAX77751 3.15A USB-C Autonomous Charger for 1-Cell Li+ Batteries` and the footer `www.maximintegrated.com / Maxim Integrated | N`.

Left in, this is not untidiness — it is a correctness failure. A query about battery chemistry matches all 40 pages equally, and the retriever has nothing to discriminate on. It also poisoned the metadata: **57 of 118 chunks in the first build had a running-header line as their section title.**

Detection is generic rather than hard-coded: normalise each text element (collapse whitespace, replace digits with `#` so page numbers unify), count the distinct pages each normalised line appears on, and drop any line appearing on ≥60% of pages. On this document that identifies 5 lines and removes 226 of 852 elements. Tables and images are never dropped.

	Before	After
Chunks containing boilerplate text	70 / 118	**9 / 104**
Chunks whose section title *is* boilerplate	57	**0**

4.6 Embedding and storage

`google/embeddinggemma-300m` (308M parameters, 768 dimensions, 2048-token window). The model is prompt-conditioned and **asymmetric**:

- documents: `title: {title} | text: {content}`
- queries: `task: search result | query: {q}`

The checkpoint's default document prompt fills the title slot with the literal string `none`. Since every chunk carries a real section title, the indexer substitutes it — that slot exists for exactly this purpose.

Vectors are L2-normalised and stored in a persistent ChromaDB collection with `hnsw:space=cosine`. Chroma metadata accepts only scalars, so `ocr_keywords` is flattened to a comma-joined string.

The model is gated on HuggingFace; the licence must be accepted and the machine authenticated (`hf auth login`) before the weights will download.

---

5. Results

5.1 Chunk inventory

Two builds are reported: **v1** (OCR'd tables, boilerplate retained) and **v2** (text-layer tables, boilerplate stripped). v2 is the current system.

Type	v1	v2
text	53	39
table	19	19
image	46	46
Total	**118**	**104**

The text-chunk count falls because 226 boilerplate elements no longer pad out the prose stream. Pages 1–40, all 40 distinct pages represented in both. Outputs: `sample_full.json` (v1), `sample_v2.json` (v2).

5.2 Runtime, latency and hardware

Measured with `benchmark.py` on the machine in section 3 (RTX 3050 Laptop, 4 GB VRAM; 16 logical CPU cores) and against the deployed Cloud Run instance (4 vCPU / 16 GiB, no GPU).

Index-building — paid once per document, cached thereafter.

Stage	GPU	CPU	Speed-up
`hi_res` partition, 40 pages	—	~15 min	none — see note
Figure captioning, 46 images (LLaVA 0.5B)	~4 min (5.2 s/image)	193 s **per image**	~37x
Embedding 104 chunks (EmbeddingGemma-300m)	**9.9 s** (95 ms/chunk, 10.5/s)	71.3 s (685 ms/chunk, 1.5/s)	**7.2x**
Model load, EmbeddingGemma	17.6 s	10.3 s	—

Partitioning does not benefit from the GPU: `unstructured`'s layout model runs through the CPU build of `onnxruntime`, so the 15 minutes is CPU-bound regardless of what hardware is present.

Query-time latency — median over 8 queries, whole collection scored.

Component	GPU	CPU
Vector search (encode query + cosine over 104)	111.2 ms	121.1 ms
Keyword search (BM25)	0.0 ms	1.0 ms
Normalise and fuse	0.0 ms	0.0 ms
End to end, in process	**116.9 ms**	**119.2 ms**

The GPU buys nothing at query time. Encoding one short query string dominates, and the cosine against 104 vectors is trivial either way — 117 ms versus 119 ms is noise. BM25 costs a millisecond. This is the measurement that justifies deploying to CPU-only Cloud Run: the GPU matters when *building* an index, not when serving one.

Deployed latency, measured against the live service:

	Time
Search, warm	0.5 – 0.9 s
Search, cold	66 – 77 s (downloads 1.26 GB EmbeddingGemma)
Chat, warm	33 – 55 s
Chat, cold	80 – 137 s (adds 3.1 GB Qwen download)
`/api/pdf/sample`	2.0 s for 1.8 MB

The gap between 117 ms in-process and 0.5–0.9 s over HTTP is FastAPI overhead, JSON serialisation of the evidence snippets, and network round-trip.

Chat is slow on any timescale because a 1.5B model in fp32 on 4 vCPUs spends most of its time on prompt processing, not generation. On the local GPU the same call takes ~23 s.

Memory. EmbeddingGemma peaks at **1.89 GiB** of VRAM, which is why it fits the 4 GB card alongside everything else. LLaVA-1.5-7B in fp16 needs roughly 14 GB and does not fit even in 4-bit once the vision tower and activations are resident — the constraint that forced the 0.5B captioner and, through it, the caption quality limits in section 7.1.

5.3 The evaluation set

An earlier 10-query set proved unusable: one query was worth 10 percentage points, and its labels were written after reading the generated chunks, so it partly measured whether retrieval could find text it was derived from.

The current set (`eval_gold.yaml`) is **45 queries authored from the source PDF's page text**, with every label grounded by locating the answering term on the page:

- **Graded relevance** — 2 = page contains a complete answer, 1 = supporting mention. This makes nDCG meaningful: did we rank the complete answer above the passing mention?
- **12 queries held out** and never inspected while choosing anything.
- **Tagged by kind** — word (15), symbol (10), table (10), figure (6), multi (4) — because these classes fail for different reasons and an average hides that.

Scoring follows the task statement (*"identify all relevant pages"*): retrieved chunks are collapsed to a ranked **page** list, so `R@k` is the fraction of gold pages recovered, not merely whether one appeared.

5.4 Retrieval results

All 45 queries, α = 0.6 (vector 60% / BM25 40%):

Build	method	hit@1	hit@3	R@5	R@10	nDCG@5
v1	vector	51%	76%	67%	79%	0.597
v1	bm25	47%	64%	57%	77%	0.478
v1	hybrid	62%	82%	71%	83%	0.645
v2	vector	60%	87%	71%	80%	0.652
v2	bm25	53%	71%	63%	80%	0.545
v2	hybrid	69%	91%	78%	84%	0.710
v3	**hybrid + section**	**69%**	**93%**	**82%**	**87%**	**0.728**

v3 adds the image-chunk rebuild (§4.4) and section expansion (§5.7). Cumulative v1 → v3: **+7 hit@1, +11 R@5, +0.083 nDCG@5**, every point of it from correcting what the index *contained* rather than from reweighting what was already in it.

Holdout split (12 queries, never tuned against) confirms it rather than contradicting it: hybrid 75% hit@1, 92% hit@3, 83% R@5.

Effect of the two extraction fixes, by query kind (hybrid):

kind	n	hit@1 v1 → v2	nDCG@5 v1 → v2
symbol	10	70% → **100%**	0.527 → **0.830**
word	15	47% → **67%**	0.599 → **0.691**
table	10	80% → 70%	0.848 → 0.803
figure	6	50% → 50%	0.655 → 0.667
multi	4	75% → **25%**	0.587 → **0.314**

The two fixes hit exactly the classes they targeted. Text-layer tables restored the symbol vocabulary (`IBATT_Q`, and the numeric limits alongside it), taking symbol queries to a perfect hit@1. Boilerplate stripping lifted paraphrased word queries by 20 points — those were the queries every page matched equally.

The `multi` regression is discussed in §7.7.

5.5 Fusion weight

α sweep on the tuning split shows a broad plateau rather than a sharp optimum:

```
α:     0.0   0.2   0.4   0.5   0.6   0.7   0.8   0.9   1.0
hit@1  45%   52%   52%   58%   61%   64%   61%   58%   48%
nDCG   .468  .541  .576  .599  .619  .628  .623  .596  .564
```

Anything in 0.5–0.8 performs equivalently; the difference between 0.6 and the nominal peak at 0.7 is a single query. α = 0.6 was retained.

5.6 Two retrieval experiments that did not work

Both were implemented, measured against the same gold set with the holdout intact, and rejected. They are recorded because the negative result is informative.

HyDE (hypothetical document embeddings). `Qwen/Qwen2.5-1.5B-Instruct` writes the passage it thinks answers the query; that passage is embedded with the *document* prompt and searched, fused with the query-side hybrid score.

all 45	hit@1	R@5	R@10	nDCG@5
hybrid	**69%**	**78%**	84%	**0.710**
+ HyDE β=0.5	64%	75%	87%	0.684
HyDE only	36%	60%	77%	0.476

It generated the right vocabulary — asked about a "resistor" it produced `R_TOPOFF`, bridging the word→symbol gap it was chosen for — but the passages are generic and factually wrong (`V_CHGIN = ±12V`; the real range is 4.5–13.7V). On a corpus where the difficulty is *discrimination*, adding plausible boilerplate that matches many

pages is the same failure as the running header. It hurt `word` queries most (hit@1 67% → 53%), the class it was intended to help. The apparent R@10 gain does not survive depth: at 20 pages, β=0 recovers 95% versus 92%.

Metadata as a third channel. Section heading + table caption + OCR keywords scored separately (dense + BM25) and fused with the content score.

all 45	hit@1	R@5	R@20	nDCG@5
content	**69%**	**78%**	**95%**	**0.710**
+ metadata γ=0.3	69%	77%	92%	0.704

The two splits disagree — holdout improves (nDCG 0.768 → 0.802), tuning degrades (0.689 → 0.668) — which is the signature of noise rather than signal. The γ sweep peaks at 0.2 on the tuning split and is not confirmed anywhere else.

Cross-encoder reranking. `BAAI/bge-reranker-base` over a 20-page pool, page score = max over its chunks. Tested on both the v2 and v3 indexes.

all 45, v3	hit@3	R@3	R@5	R@10	nDCG@5
retrieval	93%	69%	**82%**	87%	**0.728**
+ rerank blend 0.6	93%	71%	82%	**89%**	0.727

Pure reranking is actively harmful (v2: nDCG 0.716 → 0.625). The cause is measurable: the cross-encoder's scores are extremely peaked, and on the v2 index it scored *every* image chunk at ~0.000 (mean +0.001, max +0.007 across all 104 chunks), so max-pooling pushed all 11 image-only pages to the bottom of every query. Below the top few chunks the scores are indistinguishable from zero, so min-max normalising them replaces retrieval's ordering with noise.

Blending 60% retrieval / 40% reranker recovers to parity but no further, and the tuning and holdout splits disagree about the sign (tuning hit@3 91% → 94%, holdout 100% → 92%). Rejected on both builds.

Why they all failed the same way. Each helps some query kinds and hurts others, and a single global fusion weight cannot express a per-kind preference — gains and losses cancel. The extraction and image fixes (§4.2, §4.4, §4.5) worked because they restored *missing or corrupted information*; HyDE, metadata fusion and reranking only reweight information already present. Across four reweighting techniques the combined effect on nDCG@5 was under +0.01.

5.7 Section expansion (adopted)

A chunk inheriting a score from the best chunk sharing its `title`: `score = max(own, decay × best_in_section)`, skipping groups larger than `max_group`. Small but consistently non-negative: R@5 78% → 79%, nDCG 0.710 → 0.716 on v2, concentrated in `figure` queries (R@5 83% → 92%).

Two guards were necessary. `Typical Operating Characteristics` spans 27 chunks over 5 pages — a quarter of the index — so an uncapped group lets one weak hit inject five pages; at decay 1.0 with no cap, R@5 collapses from 77% to 66%. `decay = 0.5, max_group = 4` was chosen over a stronger setting because decay 0.85 costs deep recall (R@20 95% → 92%), and pool recall is the thing not to spend.

---

6. Verification

Measured, not asserted — every figure below comes from an automated check over the full output rather than inspection of samples.

Measured on the current **v2** build (104 chunks):

Property	Method	Result
Hard size limit	all 104 chunks vs 5000 + 500	**0 violations**
Min-size rule	text chunks < 1000 chars	**0** (min 1020)
Text→text overlap	longest suffix/prefix match	**38/38 pairs, 499–500 chars**
Image/table overlap	prefix byte-compared to previous chunk's tail	**65/65 exact**
`core_text` integrity	reconstruct `text` minus prefix	**65/65 exact**
Markdown validity	header + separator + column counts	**19/19 valid, 0 ragged rows**
Page + title present	all chunks	**104/104**
Base64 decodes	all image chunks rendered in a browser	**46/46, 0 failures**
Chroma round-trip	re-read metadata after insert	base64, keywords, `core_text` intact
Provenance	cache key = SHA-256(path+size+mtime) vs source	matches the supplied PDF; pages 1–40 only

Header rows in v2 come through correctly where the text layer was used — e.g. `| PARAMETER | SYMBOL | CONDITIONS | MIN TYP MAX | UNITS |` on the Electrical Characteristics tables, versus the OCR'd `| VCHGIN, BYP Continuous Current .....0.......:eeeeeeeeeeees |` that survives on the 2 fallback tables.

---

7. Limitations

Stated plainly; each is a real constraint on how far this output can be trusted.

7.1 Image captions are unreliable — and measurably costly

The 0.5B captioner hallucinates on technical figures. The page-23 charger state machine is described as *"a green circle labeled 'Input' and a red circle labeled 'Output'"*; no such circles exist.

This is not cosmetic. For the query *"what resistor value sets the top-off current?"*, a pin-diagram image chunk captioned *"a black and white schematic diagram of a computer circuit board"* scored **cos 0.505**, outranking `Table 4. Top-Off Current Settings` — which contains the literal answer — at **cos 0.446**. In v2 that specific query is fixed by the table extraction work, but the underlying weakness remains and now dominates the residual failures (§7.7).

The fix is a larger captioner, which this GPU cannot host. Mitigations that do not require better hardware: lead image chunks with OCR keywords rather than the caption, and suppress captions for the 15 of 46 images where OCR returned nothing (those chunks are currently pure hallucination with no counterweight). Both were proposed and deliberately deferred.

7.2 OCR quality — fixed for tables, unchanged for figures

Reading tables from the text layer (§4.2) eliminated the worst of this: dot-leader rows rendering as runs of `e`, lost symbol columns, `VDAT_REF` → `VoatReR`. What remains is OCR over **figures**, where no text layer exists to fall back on:

- Rotated axis labels stay garbled: `anoqd`, `soyvho`, `jyvlsay`, `lroporr`.
- **15 of 46 images** produce no usable keywords at all.
- **2 of 19 tables** still fall back to OCR, where pdfplumber found no counterpart.

7.3 Table structure

- Header rows are taken as row 0 rather than detected. With text-layer extraction this is usually correct (`PARAMETER | SYMBOL | CONDITIONS | MIN TYP MAX | UNITS`), but a table whose first row is data would repeat that data row as the header of every split part.
- **Caption detection reached only 8 of 19 tables**, and one of those 8 is wrong — a package-drawing note (`(3mm x 3mm, 0.4mm PITCH)`) captured as a table caption.
- Text-layer tables are matched to `unstructured`'s Table elements **positionally within a page**. If the two disagree on how many tables a page holds, the alignment can slip; the fallback then serves OCR'd content for the remainder.

7.4 The overlap/markdown conflict

Requirements R3 (overlap everywhere) and R6 (tables as valid markdown) conflict. In **5 table chunks** the borrowed 500 characters come from another table, so the chunk's raw `text` contains a truncated fragment of a foreign pipe-table above its own. `core_text` is always clean; only the concatenated `text` is affected. This is an open design decision, not a defect — the candidate resolutions are to strip pipe rows from the borrowed tail, or to move the overlap into metadata for table chunks only.

7.5 Untested code paths

No table in this document exceeded 5000 characters and no text chunk exceeded the 3000-character soft limit. The table-splitting logic — header repetition and row-level overlap — is therefore verified **only against a synthetic 27,000-character table** (6 parts, each ≤5000 chars, ~408 chars of shared rows between consecutive parts), never against real input.

7.6 The evaluation is still author-written

The 45-query set is a large improvement on its 10-query predecessor — authored from the source PDF rather than the chunks, graded, with a holdout split — but the queries and their labels were still written by the same person who built the system. It is not an independent benchmark. At 45 queries one case is worth ~2 points, so differences of a few points should not be treated as meaningful, and the 12-query holdout split moves ~8 points per query.

7.7 The `multi` regression (resolved in v3)

Resolved. Restructuring the image chunks (§4.4) recovered these queries: `multi` hit@1 25% → 75%, R@5 38% → 62%, nDCG 0.314 → 0.629, and `figure` R@5 83% → 100%. Pages 34–35 now carry `Figure 12/13. System Configuration for Temperature Management` — the authors' own caption — instead of an invented one.

The trade is real and worth stating: `word` R@5 fell 73% → 69% and `table` nDCG 0.798 → 0.758, because figure pages are now competitive enough to take rank-1 slots they previously could not reach. Net across all 45 queries this is positive (nDCG 0.716 → 0.728), but it is a redistribution, not a free gain.

The original diagnosis, retained for the record:

7.7b Original analysis of the regression

Multi-section queries fell from 75% to 25% hit@1 between v1 and v2. With **n = 4** that is three queries becoming one, so the magnitude is not trustworthy — but the direction has a traceable cause. Both failing queries target pages 34–35 (*System Configuration for Temperature Management*), which are dominated by Figures 12 and 13 with little body text. Those pages are represented in the index mainly by image chunks, and therefore by hallucinated captions (§7.1).

In v1 those pages likely ranked higher incidentally, via boilerplate and OCR noise that gave lexical matching spurious surface area. Removing that noise made the genuine weakness visible rather than creating it. This is the clearest evidence that figure captioning is now the binding constraint on retrieval quality.

7b. Application and deployment

The served output

The task asks for pages and, optionally, section headings. The API returns exactly that: chunk hits are collapsed to a ranked page list (first occurrence of each page wins), each page carries the section headings of the chunks that matched, plus the evidence snippets that caused the match. Pages scoring within 55% of the best page are returned, capped at 8 — a fixed top-k would be wrong in both directions, since "all relevant pages" varies per query.

The UI puts a PDF.js viewer beside the query panel; selecting a result opens that page. A second tab answers in prose from the retrieved pages using a local `Qwen/Qwen2.5-1.5B-Instruct`, with clickable page citations (the bonus task).

Deployment

Cloud Run, `us-central1`, 1 instance × 4 vCPU / 16 GiB, scaling to zero. `HF_TOKEN` is injected from Secret Manager at runtime and appears in neither the image nor the repository. CI runs the unit tests on every push and pull request and deploys only on pushes to `main`, so a fork PR can never reach the credentials.

Measured against the live URL:

Query, warm	0.54 – 0.67 s
Query, cold	74 s (1.26 GB model download)
Chat, cold	137 s (3.1 GB download + CPU generation)
Reference query result	p17 first at 0.922, matching local exactly
Chat answer	"The charger supports Li-ion and Li-Polymer batteries." (p17, p19)

Three deployment limits, all properties of the hosting rather than the pipeline: cold starts re-download weights because they are not baked into the image; chat runs a 1.5B model in fp32 on CPU and is correspondingly slow; and uploading a new PDF does not complete in the cloud, because ~20 minutes of indexing happens on a background thread with no request holding the instance alive. Upload works locally. Fixing it properly needs a job queue rather than a flag.

An earlier deployment to a Google Workspace organization had to be abandoned: `constraints/iam.allowedPolicyMemberDomains` restricts every IAM principal to the Workspace customer, which blocks not only `allUsers` but any external reviewer account, and the constraint is only changeable at the organization level.

---

8. Reproduction

```
pip install -r requirements.txt
```

System binaries required: `tesseract`, `poppler`. GPU users must pin an explicit CUDA torch build (see `SETUP.md`).

```
hf auth login          # EmbeddingGemma is gated
```

```
python pdf_chunker.py sample.pdf --model llava-hf/llava-interleave-qwen-0.5b-hf --device cuda --cache-dir .cache --out chunks.json
```

```
python embed_index.py chunks.json --db ./chroma --collection max77751_v2 --device cuda --reset
```

```
python hybrid_search.py "what does the STAT pin indicate?" --alpha 0.6 -k 5
```

```
python -m evaluation.eval_v2 --collection max77751_v2 --by-kind
```

Boilerplate stripping and text-layer tables are on by default; `--keep-boilerplate` and `--tables-from ocr` reproduce the v1 behaviour for comparison.

---

9. Files

File	Purpose
`pdf_chunker.py`	Partition → text/table/image chunks, boilerplate stripping, text-layer tables
`embed_index.py`	Embed with EmbeddingGemma → ChromaDB
`query_index.py`	Dense-only query (applies the query prompt)
`hybrid_search.py`	Dense + BM25 fusion, min-max normalised
`evaluation/eval_gold.yaml`	45 graded queries, 12 held out
`evaluation/eval_v2.py`	hit@k / R@k / nDCG@5, split by kind and by holdout
`requirements.txt`, `docs/SETUP.md`	Dependencies and setup
`sample_full.json` / `sample_v2.json`	v1 / v2 chunks
`chroma/`	Persistent vector store (`max77751`, `max77751_v2`)

---

10. Deployment

Deployed as a web application on **Google Cloud Run**.

URL	https://relevant-section-identification-qqgcuwkcmq-uc.a.run.app
Project / region	`rsi-demo-0437` / `us-central1`
Instance	1 × 4 vCPU / 16 GiB, scales to zero
Image	built from `Dockerfile` by Cloud Build, 0.97 GB compressed
Secrets	`HF_TOKEN` injected from Secret Manager at runtime, never in the image or repo
CI/CD	`.github/workflows/deploy.yml` — tests on every push and PR, deploy only on push to `main`

Verified against the live service rather than locally:

Query, warm	0.54 – 0.67 s
Query, cold	74 s (includes a 1.26 GB model download)
Chat, cold	137 s (3.1 GB download plus CPU generation)
`/api/pdf/sample`	1,819,441 bytes, byte-identical to the source
Reference query	pages 17 and 19 returned first and second
Chat answer	"*The charger supports Li-ion and Li-Polymer batteries.*"

Three limits belong to the deployment, not the pipeline:

- **Cold starts.** Weights are not baked into the image, so a reclaimed instance re-downloads them. Baking them requires Cloud Build to fetch gated weights via a build-time secret; judged disproportionate for a demo.
- **Chat is slow** — a 1.5B model in fp32 on 4 vCPUs. Correct, not fast.
- **Uploading a new PDF does not complete in the cloud.** Parsing plus figure captioning is ~20 minutes of CPU on a background thread with no request holding the instance open, so Cloud Run may reclaim it mid-index. Upload is a local feature; the cloud demo path is the pre-indexed sample.

---

11. Next steps

Both items originally listed here — cross-encoder reranking and the figure-caption rebuild — have since been done. Reranking was measured and rejected (§5.6); the caption rebuild was adopted and is the v3 build (§4.4). What remains:

1. **Bake model weights into the image** so cold starts don't re-download 4.4 GB. Needs a `cloudbuild.yaml` with build-time secret plumbing.
2. **Move indexing to a job queue** (Cloud Tasks + Cloud Run Job) so PDF upload completes in the cloud rather than only locally.
3. Resolve the overlap/markdown conflict for table chunks (§7.4).
4. Obtain queries authored by someone who has not seen the chunks (§7.6) — the single biggest weakness in the numbers reported here.
5. Exercise table splitting against a document containing a table over 5000 characters (§7.5), which this document never triggers.
6. Detect header rows rather than assuming row 0 (§7.3).

---

12. Summary of what moved the numbers

change	Δ nDCG@5	verdict
Table text from the PDF text layer + boilerplate stripping	**+0.065**	adopted
Image chunks rebuilt (figure caption first, generated caption trimmed)	**+0.012**	adopted
Section expansion via `title`	+0.006	adopted
HyDE (Qwen2.5-1.5B)	−0.026	rejected
Metadata scoring channel	−0.006	rejected
Cross-encoder reranking (best blend, two index builds)	−0.001	rejected

Three techniques that reweight signal already present in the index contributed +0.006 between them. Two changes that repaired what the index *contained* contributed +0.077. On this document, retrieval quality was never a ranking problem — it was an extraction problem that looked like one.